

PART V — TOKENS, CONTEXT, AND MODEL CHOICE

Chapter 26

Prompt Engineering for Small Models in Long Loops

“Where an instruction sits in the prompt is not a formatting choice. For a small model under recency bias, it is closer to deciding whether the instruction exists at all.”

Chapter 26 — Prompt Engineering for Small Models in Long Loops

Every technique in this chapter answers the same underlying question: given a fixed, limited instruction-following budget (Chapter 25.1's working-memory ceiling), how should a prompt be structured, ordered, and trimmed so that the budget is spent on the instructions that matter most, rather than divided evenly — and therefore thinly — across everything the prompt happens to contain?

26.1 Section ordering matters

Definition — Recency bias (prompting)

The tendency of a transformer's next-token prediction to weight tokens near the end of the input more heavily than tokens further back, meaning the instructions placed closest to the generation point receive disproportionate influence over the immediate next action.

ADR-012 states the ordering principle this project settled on directly: role and rules first, contextual injections (blocked variants, thumbdown history) in the middle, and PROCESS instructions last — appended immediately before the user's actual message, the position closest to the point where the model must decide its next action. This is not an arbitrary convention; it is a deliberate exploitation of the exact mechanism Chapter 25.2 described as a liability, turned into a tool.

The ordering also encodes a second, quieter principle: put content that is stable across every query first (role, rules, tool descriptions never change turn to turn), and put content that varies by query — and therefore needs to be actively re-read each time — last. Static content the model has effectively "seen" in every training-adjacent system prompt benefits less from recency placement than instructions specific to the current call.

It is worth being precise about what recency-based ordering does and does not fix. It does not shrink the total token count a small model must process, and it does not repair Chapter 25's underlying working-memory limitation. What it does is spend the model's existing, fixed attention budget more deliberately — placing the highest-value instruction where the model's own architecture already weights attention most heavily, rather than leaving that placement to whatever order a prompt happened to accumulate its sections in over successive edits.

26.2 The before-and-after of the prompt split

The original design, per Status.md's 2026-05-12 entry, was a single monolithic ``_BASE_SYSTEM_PROMPT`` block with PROCESS instructions embedded in the middle. ADR-012 replaced it with two named constants — ``_ROLE_AND_RULES`` and ``_PROCESS_INSTRUCTIONS`` — assembled by ``_build_system_prompt()`` with everything else sandwiched between them in a specific, commented order.

```

parts = [_ROLE_AND_RULES]
if blocked_variants:
    parts.append(blocked_block)
if prior_thumbsdowns:
    parts.append(thumbsdown_history_block) # passive: what went wrong
# PROCESS goes after the history context and before the active priority
block
parts.append(_PROCESS_INSTRUCTIONS)
if prior_thumbsdowns:
    parts.append(active_priority_block) # active: what to seek — last

```

Notice the final wrinkle: when thumbsdown feedback exists, an *active priority block* is appended after ``_PROCESS_INSTRUCTIONS``, not before it. This is deliberate — the block restating what the user flagged as wrong needs to be even closer to the generation point than the general PROCESS steps, because it is the single most query-specific, highest-priority instruction the prompt can contain for that particular retry.

26.3 Strengthening "do NOT batch tool calls"

``_PROCESS_INSTRUCTIONS`` does not merely state its four-step procedure — it labels itself explicitly: "PROCESS (follow in this exact order — do NOT batch tool calls)", then numbers each step and marks GOOD/BAD examples inline for step 1 specifically, the step most prone to the premature-batching failure ADR-012 was written to stop.

```

1. FIRST, call retrieve_documents 2-3 times with SHORT, semantically
different queries.
    Wait for results before doing anything else. Do NOT call
compress_context yet.
    GOOD: short noun-phrase queries from different angles of the topic.
    BAD: rephrasings of the same angle ("what causes X", "X causes",
"causes of X") — forbidden.

```

Numbering the steps and naming the anti-pattern explicitly ("do NOT batch tool calls") rather than describing the desired behavior only in the positive ("call tools one purpose at a time") is itself an application of Chapter 25.5's finding: a small model responds more reliably to an explicitly named failure mode than to an implicit contrast it must infer.

26.4 Compressing blocked-variants and thumbsdown sections — the token-cost math

ADR-013 records the number that makes this section necessary: thumbsdown history injection reached "up to ~1,800 tokens in Run 3" — coincidentally very close to the entire instruction-following ceiling ADR-013 separately measured for the 8B model. A contextual injection block that grows unboundedly can, on its own, consume the model's entire reliable instruction-following budget before ``_PROCESS_INSTRUCTIONS`` is even reached.

The math is straightforward per record: each injected prior-failure entry in ``_build_system_prompt()`` includes the original query, the user's feedback text, up to a 300-character bad-answer snippet, every

reformulation tried, and up to two retrieved-chunk previews (200 characters each) per reformulation. A single richly-populated thumbdown record can easily run 400-600 tokens; three or four such records, uncapped, reconstructs ADR-013's 1,800-token problem from a handful of prior sessions alone.

This is the same per-item budgeting discipline Chapter 23.4 applied to retrieved chunks, now applied to a different kind of injected content. Both cases share the identical shape: a per-item token cost, multiplied by however many items a naive design would inject without an explicit cap, compared against a measured ceiling the model respects. Once that comparison is made explicit, the cap stops looking like an arbitrary conservatism and starts looking like the only number consistent with the model's own measured limits.

26.5 Capping how many prior-failure records to inject

ADR-013's stated remedy is a cap: "Cap on thumbdown injection (most recent 2) recommended to prevent prompt bloat beyond the 8B model's instruction-following ceiling." This is Section 26.4's token math translated directly into a policy — not "inject everything relevant," but "inject only as much as the measured ceiling can actually absorb without crowding out `_PROCESS_INSTRUCTIONS``."

Common pitfall — Optimizing for recall over budget

Injecting every prior thumbdown record feels like it should help — more history, more context to avoid repeating a mistake. Past the instruction-following ceiling, additional records actively hurt, by diluting attention away from the procedural rules the model needs most. Two well-chosen records reliably obeyed beat five records that push PROCESS instructions past the model's effective ceiling.

Choosing "most recent" as the selection criterion for which records survive the cap, rather than some notion of "most similar" or "most severe," is itself a deliberate simplicity choice: recency of the thumbdown event is trivially computable from the record's own timestamp, requires no additional judge call to rank by relevance, and in practice correlates reasonably well with what a user is likely to still consider an open, unresolved complaint about this exact question.

26.6 Shorter tool schema descriptions reclaim instruction-following budget

Tool schema descriptions are easy to overlook as a budget line item because they are not part of the system prompt text itself — but they are still tokens the model reads on every call. This project's own ``retrieve_documents`` and ``compress_context`` descriptions in ``tools.py`` each restate procedural rules — "Call this 2-3 times first," "Call this EXACTLY ONCE, AFTER your ``retrieve_documents`` calls are done" — that ``_PROCESS_INSTRUCTIONS`` already states in full, numbered detail.

This duplication is worth naming honestly rather than presenting as already resolved: it is a real, present opportunity in this codebase, not a shipped optimization. A tool description's job is to convey what the tool does and what arguments it needs — the **when** and **how many times** belong in ``_PROCESS_INSTRUCTIONS``, stated once, where Section 26.1's recency placement already gives it maximum weight. Trimming each tool description to its minimal, non-duplicated form would reclaim real tokens from a section of the prompt the model reads on every single call, without removing any rule the model does not already receive elsewhere.

There is a second cost to the duplication beyond raw token count, worth naming for the same reason Chapter 25.3 flagged conflicting soft instructions as a risk: two descriptions of the identical rule, worded slightly differently in the tool schema versus ``_PROCESS_INSTRUCTIONS``, create an opportunity for the two to drift out of sync as either one is edited independently over time. A single source of truth for a procedural rule — stated once, in the place recency bias weights most heavily — is not only cheaper in tokens but also safer against the two descriptions silently disagreeing after a future edit touches one and not the other.

26.7 Demonstration over description

Several of this project's own prompts do not merely describe the desired output shape — they show it. `\_CHUNK\_MERGE\_PROMPT` and `\_DC\_SCAN\_PROMPT` both include a full worked EXAMPLE INPUT paired with the exact EXAMPLE OUTPUT expected, using the project's own ASD-domain content as the example material rather than a generic placeholder.

```
EXAMPLE INPUT:
[Source: a.pdf]
ASD affects 1 in 36 children. Early diagnosis improves outcomes.

EXAMPLE OUTPUT:
{"content": "ASD affects approximately 1 in 36 children [Source:
a.pdf]...",
 "sources": ["a.pdf"], "merged_from": 2}}
```

A worked example is a stronger signal than a schema description for the identical reason Chapter 25.3 gave for preferring architectural constraints over soft instructions: describing a JSON shape in prose asks the model to translate a rule into an output; showing the exact shape asks the model to pattern-match, which is closer to what next-token prediction already does well. This is precisely the mechanism available to fix Chapter 25.5's text-vs-code mode-flip failure at its root, rather than only patching it after the fact with a negative instruction.

26.8 Adding negative instructions when small models drift

Chapter 25.5 already showed the artifact: "You are NOT writing software. You are NOT generating Python. You are NOT solving a coding task." repeated verbatim across `\_DC\_SCAN\_PROMPT` and `\_REDUNDANCY\_JUDGE\_PROMPT`. This section states the general rule that specific fix instantiates: when a small model's failure mode has a name — a specific wrong output format, a specific premature action — state that failure by name and forbid it explicitly, rather than trusting a positive instruction to rule it out implicitly.

The two techniques from Sections 26.7 and 26.8 are not competitors; this project's own prompts use both together. `\_DC\_SCAN\_PROMPT` pairs its negative instruction with worked GOOD/BAD redundancy examples in the same prompt. A positive demonstration shows the model what correct output looks like; a negative instruction rules out the single most likely wrong path a small model drifts toward under pressure. Neither alone reliably prevents both failure directions.

Figure 26.1 draws together every ordering decision from Sections 26.1 through 26.5 into a single picture: it is the actual layout `\_build\_system\_prompt()` assembles, top to bottom, with each block's recency-driven placement made visually explicit rather than left implicit in the function's control flow.

Prompt anatomy: closest to generation wins recency bias

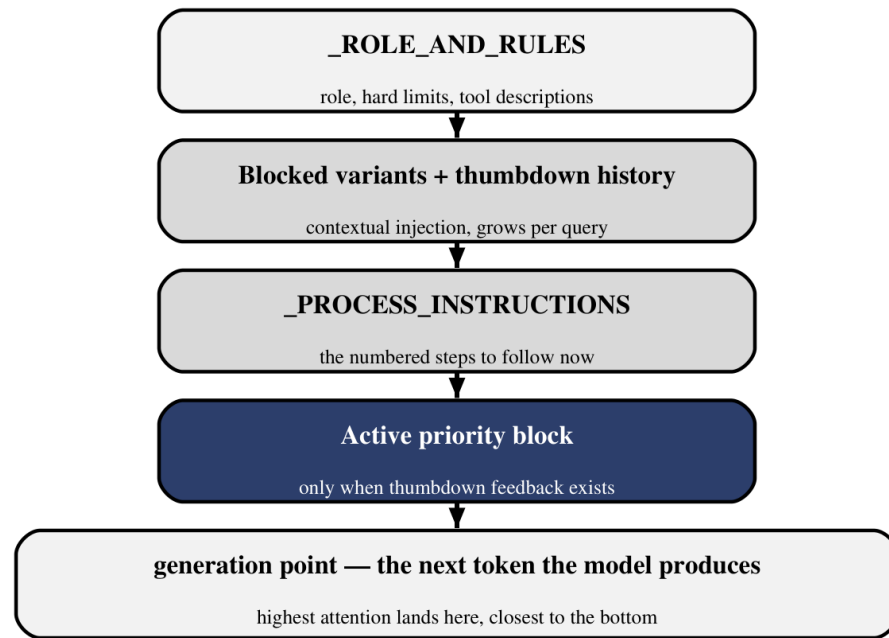


Figure 26.1 — Static role and rules lead; query-specific context sits in the middle; the exact procedure to follow now sits last, closest to the generation point.

26.9 Measuring the fix

ADR-012's own stated impact is concrete and falsifiable, not merely theoretical: the split prompt structure, combined with the OUTPUT FORMAT constraints added afterward, was a direct response to "the 11,133-char repetition-degeneration failure observed when context dilution caused the model to vomit citations in a loop." The before/after in Figure 26.2 is not a hypothetical comparison — it is this project's own actual prompt-architecture history, one version replaced by another after a specific, logged failure.

Monolithic prompt vs. recency-split prompt

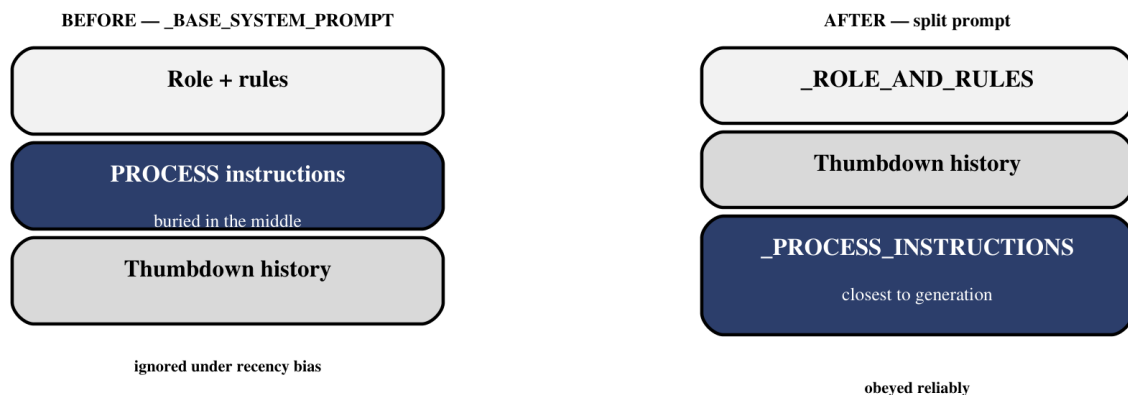


Figure 26.2 — The monolithic prompt buried PROCESS instructions where recency bias could not reach them; the split prompt places them last, deliberately.

The honest caveat worth stating: none of these techniques change what the model fundamentally is. They change where the model's limited, probabilistic attention is spent, given that it will always be limited and probabilistic for a model this size. Chapter 25.3's distinction — soft instruction versus hard architectural constraint — still applies underneath every technique in this chapter; a well-placed instruction is more likely to be obeyed, never guaranteed to be.

Prompt structure alone cannot substitute for state the system itself must track reliably regardless of what the model reads or ignores on a given turn. The next chapter turns to exactly that: the state-engineering mechanisms — tool-result injection, hard filters, history scrubbing — that keep an agentic loop correct even when a small model's attention to any single instruction cannot be fully guaranteed.

Taken together, Sections 26.1 through 26.8 describe a single coherent discipline rather than eight independent tricks: identify what the model most needs to obey right now, place it where recency bias gives it maximum weight, keep everything sharing that budget as small as the measured ceiling allows, and prefer showing or explicitly forbidding over merely describing when a specific failure mode has already been observed. Every one of this project's own prompt-engineering decisions in ``prompts.py`` and ``agent_query.py`` traces back to one of these four moves, applied to a specific, logged failure rather than to a generic best practice borrowed unmodified from elsewhere.

This is also why the chapter opened with a caveat worth repeating at the close: none of these techniques are a substitute for the architectural constraints Chapter 25.3 already argued for. A well-ordered, well-trimmed prompt makes an 8B model's **soft** instruction-following meaningfully more reliable — Section 26.9's before/after is real evidence of that — but it remains soft. Any constraint where an occasional violation would be genuinely costly still belongs one layer down, enforced in code rather than merely well-placed in the prompt the model happens to read.